SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

C PROGRAMMING ASSIGNMENT

Student Submission

Student Name	D bharath
Register Number	1972260104
Course Code & Name	CSA0208-c programming
Slot	A
Assignment / Task Title	Electricity Bill Calculator — Tasks 1 to 4
GitHub Repository URL	https://github.com/bharathdasaraju01-droid/CSA-0208-C-Programming.git
Submission Date	28 Oct. 26

Instructions to Students

1. Complete every section for each assigned task.
2. Write your own explanation, algorithm/pseudocode, and program.
3. Run the program and insert a clear screenshot of the output.
4. Include test cases and a short analysis where applicable.
5. If code is maintained on GitHub, provide the correct repository link.

About the Program Used in All Four Tasks

All tasks use one program, an **Electricity Bill Calculator**. It reads a consumer number, name, category (D = domestic, C = commercial) and the units consumed, then prints a bill. **All money is stored as an integer number of paise** (100 paise = Rs 1) so that no floating-point error can creep in.

Item	**Rule**
Slab 1	Units 1–100 at Rs 1.50 per unit
Slab 2	Units 101–200 at Rs 2.50 per unit
Slab 3	Units above 200 at Rs 4.00 per unit
Fixed charge	Rs 25.00 (domestic) or Rs 100.00 (commercial)
Tax	5% of (energy + fixed), rounded half-up to the nearest paisa
File	**Purpose**
`bill.h`	Constants, DEBUG macro, function prototypes (include guard)
`input.c`	read_record(): scanf-based input with validation
`tariff.c`	energy_charge_paise(), fixed_charge_paise(), tax_paise()

`report.c`	print_paise(), print_bill()
`main.c`	main(): ties everything together
`bill.c`	The same program as one single file (used in Tasks 1–4)

TASK 1 — Organise the Program with Header Files and Conditional Compilation

3 Marks | Bloom's Level: K3 – Apply

Task / Problem Statement

Move the constants and function prototypes of the electricity-bill program into a header file, protect the header with include guards, and add a debugging mode that prints intermediate values only when the program is compiled with `gcc -DDEBUG`.

Aim

To organise a C program into a header file plus source files, and to use the preprocessor (include guards and `#ifdef DEBUG`) to control what is compiled.

Concept / Theory

Structure of a C program

A C source file is normally laid out in this order: (1) **preprocessor directives and header files** (`#include`, `#define`); (2) **global declarations** such as constants, global variables and function prototypes; (3) the **main()** function, where execution starts; (4) **other functions** that main() calls. In this program the first two parts live in `bill.h`, and the functions live in the .c files.

Include guards

A header can be included more than once, for example directly and again through another header. Declarations such as `struct` or `typedef` would then be defined twice and the compiler reports a redefinition error. The guard `#ifndef BILL_H / #define BILL_H / ... / #endif` makes the body count only the first time: the second time `BILL_H` is already defined, so the preprocessor skips everything up to `#endif`.

Conditional compilation

The macro `DBG(...)` is defined as a call to `fprintf(stderr, ...)` only `#ifdef DEBUG`. Compiling with `gcc -DDEBUG` defines DEBUG before preprocessing starts. Without it, `DBG(...)` becomes `((void)0)`, which generates no code at all.

#include <file> versus #include "file"

The angle-bracket form searches only the system include directories (such as /usr/include). The quoted form searches the directory of the current source file first, then falls back to the system directories. So `<stdio.h>` is used for standard headers and `"bill.h"` for the programmer's own headers.

Procedure / Algorithm

1. Collect all tariff constants and the DBG macro into `bill.h` and wrap the file in `#ifndef BILL_H` ... `#endif`.
2. Write the prototypes of every function in `bill.h`; include it in every .c file with `#include "bill.h"`.
3. Place `DBG("...")` calls at the points where intermediate values are interesting (units read, energy charge, tax).
4. Compile normally (`gcc bill.c`) and run; no debug lines appear.
5. Compile with `gcc -DDEBUG bill.c` and run; the debug lines appear on stderr.
6. Test the include guard by including a header twice, with and without the guard.

Program / Source Code

bill.h

```
#ifndef BILL_H
#define BILL_H

#include <stdio.h>

/* ---- Tariff constants (all money in paise: 100 paise = Rs 1) ---- */
#define SLAB1_LIMIT          100L   /* units 1-100    */
#define SLAB2_LIMIT          200L   /* units 101-200  */
#define RATE1_PAISE          150L   /* Rs 1.50 / unit */
#define RATE2_PAISE          250L   /* Rs 2.50 / unit */
#define RATE3_PAISE          400L   /* Rs 4.00 / unit */
```

```
#define FIXED_DOMESTIC      2500L   /* Rs 25.00       */
#define FIXED_COMMERCIAL   10000L   /* Rs 100.00      */
#define TAX_PERCENT            5L
#define MAX_UNITS         100000L
#define NAME_LEN               50

/* ---- Debugging mode: compile with  gcc -DDEBUG ... ---- */
#ifdef DEBUG
#define DBG(...) fprintf(stderr, "[DEBUG] " __VA_ARGS__)
#else
#define DBG(...) ((void)0)
#endif

/* ---- Function prototypes ---- */
int  read_record(unsigned int *id, char *name, char *category, long *units);
long energy_charge_paise(long units); long fixed_charge_paise(char category);
long tax_paise(long subtotal_paise); void print_paise(long paise); void
print_bill(unsigned int id, const char *name, char category,
long units, long energy, long fixed, long tax); #endif /* BILL_H */
```

tariff.c *(uses DBG)*

```
#include "bill.h"

long energy_charge_paise(long units)
{
    long charge = 0, s;

    s = (units < SLAB1_LIMIT) ? units : SLAB1_LIMIT;     charge += s *
RATE1_PAISE;     if (units > SLAB1_LIMIT) {         s = ((units <
SLAB2_LIMIT) ? units : SLAB2_LIMIT) - SLAB1_LIMIT;         charge += s *
RATE2_PAISE;
    }
    if (units > SLAB2_LIMIT)         charge += (units -
SLAB2_LIMIT) * RATE3_PAISE;

    DBG("units=%ld -> energy charge=%ld paise\n", units, charge);
return charge; }

long fixed_charge_paise(char category)
{
    return (category == 'C') ? FIXED_COMMERCIAL : FIXED_DOMESTIC; }

/* 5% tax, rounded half-up, using integers only */ long
tax_paise(long subtotal_paise)
{
    long t = (subtotal_paise * TAX_PERCENT + 50) / 100;
    DBG("subtotal=%ld -> tax=%ld paise\n", subtotal_paise, t);
return t;
}
```

main.c

```
#include "bill.h"

int main(void)
{
    unsigned int id;     char
name[NAME_LEN], category;     long
units, energy, fixed, tax;

    if (!read_record(&id, name, &category, &units)) {
fprintf(stderr, "Invalid input. Program terminated.\n");
return 1;
    }
    energy  =  energy_charge_paise(units);           fixed    =
fixed_charge_paise(category);      tax     = tax_paise(energy +
fixed);     print_bill(id, name, category, units, energy, fixed,
tax);     return 0;
}
```

Output Screenshot

(a) Normal build — no debug output

```
$ gcc -Wall -Wextra -o bill bill.c
$ ./bill
Enter consumer number : 1001
Enter consumer name   : Ravi
Category (D/C)        : D
```

```
Units consumed        : 250

========== ELECTRICITY BILL ==========
Consumer No : 1001
Name        : Ravi
Category    : D
Units       : 250
--------------------------------------
Energy      : Rs 600.00
Fixed       : Rs 25.00
Tax (5%)    : Rs 31.25
--------------------------------------
TOTAL       : Rs 656.25
======================================
```

(b) Debug build — lines starting with [DEBUG] are printed on stderr

```
$ gcc -Wall -Wextra -DDEBUG -o bill_dbg bill.c
$ ./bill_dbg
Enter consumer number : 1001
Enter consumer name   : Ravi
Category (D/C)        : D
Units consumed        : 250
[DEBUG] read units=250 on attempt 1
[DEBUG] units=250 -> energy charge=60000 paise
[DEBUG] subtotal=62500 -> tax=3125 paise

========== ELECTRICITY BILL ==========
Consumer No : 1001
Name        : Ravi
Category    : D
Units       : 250
--------------------------------------
Energy      : Rs 600.00
Fixed       : Rs 25.00
Tax (5%)    : Rs 31.25
--------------------------------------
TOTAL       : Rs 656.25
======================================
```

(c) Include guard test (a header with a struct, included twice)

```
$ gcc dg1.c        (demo.h has NO include guard)
In file included from dg1.c:2:
demo.h:1:8: error: redefinition of 'struct Point'
    1 | struct Point { int x; int y; };
      |        ^~~~~

$ gcc dg2.c        (demo_g.h WITH include guard) compiled with no errors
```

(d) "file" finds the local stdio.h first; goes to /usr/include

```
$ gcc -E q_test.c | grep "stdio.h" | head -3   (#include "stdio.h", a local stdio.h exists)
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 1 "q_test.c"
# 1 "stdio.h" 1

$ gcc -E q_test.c  (with <stdio.h> written instead)
# 1 "q2.c"
# 1 "/usr/include/stdio.h" 1 3 4
```

Analysis / Discussion

- (a) and (b) give an identical bill. In (b) three extra lines appear on **stderr**, so debug output never mixes with the bill on stdout.
- (c) Without a guard gcc stops with *redefinition of 'struct Point'*; with the guard the second inclusion is skipped and the file compiles. bill.h itself only holds macros and prototypes, which can be repeated safely, but the guard is still required good practice, since any later struct would break it.
- (d) For `#include "stdio.h"` the first file read is the local `stdio.h`; with `<stdio.h>` it is `/usr/include/stdio.h`.
- The executable compiled with -DDEBUG is larger (4888 vs 4645 bytes of text) because the debug code is really inside it.

Alternative Concept and Complexity Analysis — run-time debug flag

Solution. Replace the macro with a global variable `int debug` that is set at start-up (for example by a `-d` command-line option) and write `if (debug) fprintf(stderr, ...);` at each debug point. The debug code is

always in the program; the flag decides at run time whether it executes. This allows debugging a released build without recompiling.

Measurement. The tariff function was called 200 million times with each method (gcc -O1, three runs):

```
ifdef (DEBUG off) : 0.437 s  (sink=6306663840000) run-
time if(debug): 0.650 s  (sink=6306663840000) ifdef
(DEBUG off) : 0.523 s  (sink=6306663840000) run-time
if(debug): 0.740 s  (sink=6306663840000) ifdef (DEBUG
off) : 0.444 s  (sink=6306663840000) run-time
if(debug): 0.663 s  (sink=6306663840000)
```

```
$ size b_none.o b_rt.o b_dbg.o
   text       data        bss        dec        hex    filename
418          0          4        422        1a6    b_none.o
    495          0          4        499        1f3    b_rt.o
    483          0          4        487        1e7    b_dbg.o

$ ls -l bill bill_dbg
16648 bill
16648 bill_dbg
```

	#ifdef DEBUG (off)	Run-time if (debug)
Extra time per call	0 — no instruction exists	one load + compare + branch = O(1) per call; measured about 0.2 s / 2×10^8 ≈ 1 ns
Total extra time for k calls	0	Θ(k), small constant
Extra space	0 bytes (418 B of code in the test function)	code + format strings stay in the file: 495 B (+77 B) even when never used; grows O(d) with d debug statements
Switching debug on	Needs recompilation	Instant, no recompilation

Conclusion. The test costs only about a nanosecond, but it is paid on every call, and the debug code and messages stay in the shipped program. #ifdef costs nothing at run time but needs a rebuild to switch on. For a tight inner loop or a small embedded device #ifdef is better; for field diagnostics the run-time flag is more convenient.

Result

The constants and prototypes were successfully moved to a guarded header, and the DEBUG mode printed intermediate values only when compiled with `-DDEBUG`. I learned how include guards prevent redefinition errors, and how the preprocessor removes code that is not needed.

TASK 2 — Read and Print Data Correctly

4 Marks | Bloom's Level: K3 – Apply

Task / Problem Statement

Design the input and output of the bill program with `scanf` and `printf`: choose the correct format specifier for each data type, show what happens with a wrong specifier, validate the input using the return value of scanf, and print an integer amount in paise as rupees and paise (for example Rs 162.50).

Aim

To read the consumer record safely, reject invalid input, and print a correctly formatted bill.

Concept / Theory

Type	printf	scanf	Note
int	`%d`	`%d`	signed decimal
unsigned int	`%u`	`%u`	used for consumer number
long	`%ld`	`%ld`	units, all paise values
long long	`%lld`	`%lld`	64-bit on every platform
float	`%f`	`%f`	printf promotes float to double

double	`%f or %lf`	`%lf`	scanf needs **%lf** for double
char	`%c`	`%c`	leading space skips white-space
string	`%s`	`%49s`	width limit stops buffer overflow

- **Wrong specifier:** printf reads the argument using the size and register class the specifier names. A mismatch gives wrong or undefined output; gcc -Wall warns for some cases.
- **Return value of scanf:** scanf returns the number of items successfully converted (or EOF). If it is not equal to the number requested, the input was invalid. Characters that could not be converted stay in the input buffer and must be discarded before retrying.
- **Rupees and paise:** keep the amount in integer paise p. Then rupees = p / 100 and paise = p % 100, printed with `%ld.%02ld`. The `02` pads 5 paise to .05.

Procedure / Algorithm

1. Print a prompt and read the consumer number with `scanf("%u", &id)`; if the return value is not 1, report an error.
2. Read the name with `%49s` and the category with `" %c"`; reject any category other than D or C.
3. Loop up to 3 times: read units with `%ld`; if scanf returns 1 and 0 ≤ units ≤ MAX_UNITS accept it; otherwise print an error and discard the rest of the line.
4. If all attempts fail, stop with exit code 1.
5. Print the bill; every amount goes through print_paise(), which prints `Rs %ld.%02ld`.

Program / Source Code

input.c

```
#include "bill.h"

/* discard the rest of the current input line */ static
void flush_line(void)
{     int c;     while ((c = getchar()) != '\n'
&& c != EOF)
        ;
}

/* returns 1 on success, 0 on invalid / missing input */
int read_record(unsigned int *id, char *name, char *category, long *units)
{
    int attempts;
```

```
    printf("Enter consumer number : ");
    if (scanf("%u", id) != 1) { flush_line(); return 0; }

    printf("Enter consumer name   : ");
    if (scanf("%49s", name) != 1) return 0;     /* 49 = NAME_LEN - 1 */

    printf("Category (D/C)        : ");     if (scanf(" %c",
category) != 1) return 0;     if (*category != 'D' &&
*category != 'C') {         fprintf(stderr, "Error: category
must be D or C\n");         return 0;     }

    for (attempts = 1; attempts <= 3; attempts++) {
printf("Units consumed        : ");         if
(scanf("%ld", units) == 1) {             if (*units >=
0 && *units <= MAX_UNITS) {
                DBG("read units=%ld on attempt %d\n", *units, attempts);
return 1;
            }
            fprintf(stderr, "Error: units out of range (0-%ld)\n", MAX_UNITS);
        } else {
            if (feof(stdin)) return 0;             fprintf(stderr, "Error:
units must be a number\n");             flush_line();
/* recover from letters */
        }
    }
    return 0;
}
```

report.c

```
#include "bill.h"

void print_paise(long paise)
{
    printf("Rs %ld.%02ld", paise / 100, paise % 100); }

void print_bill(unsigned int id, const char *name, char category,
long units, long energy, long fixed, long tax)
{
    printf("\n========== ELECTRICITY BILL ==========\n");     printf("Consumer No
: %u\n", id);     printf("Name         : %s\n", name);     printf("Category    :
%c\n", category);     printf("Units        : %ld\n", units);     printf("--------
------------------------------\n");           printf("Energy             :  ");
print_paise(energy); printf("\n");    printf("Fixed     : "); print_paise(fixed);
printf("\n");      printf("Tax  (%ld%%)     : ", TAX_PERCENT); print_paise(tax);
printf("\n");                printf("-------------------------------------\n");
printf("TOTAL          : "); print_paise(energy + fixed + tax); printf("\n");
printf("=====================================\n");
}
```

spec_demo.c *— correct and wrong specifiers*

```
#include <stdio.h>
int main(void)
{
    int           i = -7;     unsigned int  u
= 3000000000u;     long          l =
5000000000L;     long long     ll =
9000000000000000000LL;     float         f =
162.5f;     double        d = 1234.56789;
char          c = 'R';     char          s[] =
"Ravi";

    printf("--- correct specifiers ---\n");
printf("%%d    int             :  %d\n",  i);
printf("%%u     unsigned     :   %u\n",   u);
printf("%%ld  long            :  %ld\n",  l);
printf("%%lld   long   long:   %lld\n",   ll);
printf("%%f    float          :  %f\n",   f);
printf("%%lf   double         :  %lf\n",  d);
printf("%%c    char           :  %c\n",   c);
printf("%%s  string    : %s\n", s);
```

```
    printf("\n--- WRONG specifiers ---\n");
printf("%%d  for long 5000000000     : %d\n", l);
```

```
printf("%%u  for int -7              : %u\n", i);
printf("%%d  for unsigned 3000000000 : %d\n", u);
printf("%%d  for double 1234.56789   : %d\n", d);
printf("%%c  for int 82 (valid)      : %c\n", 82);
return 0; }
```

Output Screenshot

(a) Format specifiers: correct, then wrong

```
$ gcc -Wall -Wextra -o spec_demo spec_demo.c
24:45: warning: format '%d' expects argument of type 'int', but argument 2 has type 'long int' [-Wformat=] 27:45:
warning: format '%d' expects argument of type 'int', but argument 2 has type 'double' [-Wformat=]
$ ./spec_demo
--- correct specifiers ---
%d  int       : -7
%u  unsigned  : 3000000000
%ld long      : 5000000000
%lld long long: 9000000000000000000
%f  float     : 162.500000
%lf double    : 1234.567890
%c  char      : R
%s  string    : Ravi

--- WRONG specifiers ---
%d  for long 5000000000     : 705032704
%u  for int -7              : 4294967289
%d  for unsigned 3000000000 : -1294967296
%d  for double 1234.56789   : 780158400
%c  for int 82 (valid)      : R
```

(b) Invalid input: letters, then a valid number

```
--- Test A: letters for units, then a valid number ---
Enter consumer number : 1001
Enter consumer name   : Ravi
Category (D/C)        : D
Units consumed        : abc
Error: units must be a number
Units consumed        : 120

========== ELECTRICITY BILL ==========
Consumer No : 1001
Name        : Ravi
Category    : D
Units       : 120
--------------------------------------
Energy      : Rs 200.00
Fixed       : Rs 25.00
Tax (5%)    : Rs 11.25 ------------------------------------
TOTAL       : Rs 236.25
======================================
```

(c) Invalid input three times

```
--- Test B: letters on all 3 attempts ---
Enter consumer number : 1001
Enter consumer name   : Ravi
Category (D/C)        : D
Units consumed        : abc
Error: units must be a number
Units consumed        : xyz
Error: units must be a number
Units consumed        : pqr
Error: units must be a number Invalid
input. Program terminated.
```

(d) Rupees and paise from integer paise

```
$ ./rp
 16250 paise  ->  Rs 162.50
     5 paise  ->  Rs 0.05
   100 paise  ->  Rs 1.00
 99999 paise  ->  Rs 999.99
     0 paise  ->  Rs 0.00
Without %02ld:  Rs 0.5  (wrong for 5 paise)
```

Analysis / Discussion

- **Wrong specifiers.** `%d` with 5000000000 prints 705032704, because only the low 32 bits are used: $5000000000 - 2^{32} = 705032704$. `%u` with −7 prints 4294967289 ($2^{32} - 7$). `%d` with 3000000000 prints −1294967296 (the same bits seen as signed). `%d` with a double prints garbage, because a double is

passed in a floating-point register and the behaviour is undefined. gcc warned only for the long and double cases.

- **Validation.** In test (b) the first scanf returns 0 for "abc", the program prints an error, discards the line and the second attempt succeeds. In (c) all three attempts fail and the program ends safely with an error message.
- **A weakness of scanf.** Input such as `12x` is accepted as 12 because scanf converts the valid prefix and leaves the "x" in the buffer; this shows the need for the stricter method below.
- **Bill.** 16250 paise prints as Rs 162.50 and 5 paise as Rs 0.05, so the bill is always exact to the paisa.

Alternative Concept and Complexity Analysis — fgets() with sscanf() / strtol()

Solution. Read a whole line into a buffer with `fgets(buf, sizeof buf, stdin)`, then convert it with `strtol(buf, &end, 10)` (or `sscanf`). The line is accepted only if `end` points to the newline, so "12x" is rejected. `errno == ERANGE` also detects overflow, which scanf leaves undefined.

Recovery from invalid input

```
$ printf "abc
250
" | ./alt2b   (fgets+strtol)
line rejected : abc line
accepted : 250

Naive scanf loop WITHOUT flushing the bad line (max 5 tries shown):
try 1: scanf returned 0, 'abc' still in buffer try
2: scanf returned 0, 'abc' still in buffer try 3:
scanf returned 0, 'abc' still in buffer try 4:
scanf returned 0, 'abc' still in buffer try 5:
scanf returned 0, 'abc' still in buffer
...would loop forever
```

Time to read and add 3,000,000 numbers (gcc -O2, three runs)

```
$ 3,000,000 lines read each way
scanf       : 3000000 values, sum=149980490808, 0.272 s fgets+strtol: 3000000 values,
sum=149980490808, 0.138 s scanf       : 3000000 values, sum=149980490808, 0.349 s
fgets+strtol: 3000000 values, sum=149980490808, 0.178 s scanf       : 3000000 values,
sum=149980490808, 0.274 s fgets+strtol: 3000000 values, sum=149980490808, 0.132 s
```

	scanf("%ld")	**fgets + strtol**
Time for a line of length L	O(L), larger constant (parses the format string, locks stdin) — measured 0.27–0.35 s	O(L) to copy + O(L) to convert — measured 0.13–0.18 s, about 2× faster
Buffer space	O(1) own buffer (stdio keeps its own internal buffer)	O(B) own buffer of B bytes (64 here); B must be ≥ L + 2 or the long line is split
Recovery from bad input	Bad characters stay in stdin; without an explicit flush the loop repeats forever (test above)	The whole line is already consumed, so the next attempt starts on a clean line
Strictness	Accepts "12x" as 12; overflow undefined	Detects trailing garbage and overflow

Result

The program reads data with the correct specifiers, validates it through the return value of scanf, and prints the bill in rupees and paise. I learned that a wrong specifier silently gives wrong values, and that fgets + strtol is safer for input that must be validated.

TASK 3 — Control Type Conversion and Precision

4 Marks | Bloom's Level: K4 – Analyse

Task / Problem Statement

Find every place in the bill program where values of different types are combined, demonstrate at least three expressions that give an unexpected result, explain each, and fix a rounding rule for converting a fractional charge to paise.

Aim

To analyse implicit and explicit type conversion in C and to show why the bill is calculated in integer paise with one consistent rounding rule.

Concept / Theory

- **Integer promotion:** char and short are converted to int before arithmetic. **Usual arithmetic conversions:** the operands are brought to a common type: if one is double the other becomes double; else float; else the lower-rank integer type is converted to the higher one; if a signed and an unsigned type of the same rank meet, the *signed value becomes unsigned*.
- **Integer division** discards the fraction: 250 / 100 is 2, not 2.5.
- **Explicit cast** (type)expr forces a conversion. Casting a floating value to an integer *truncates*; casting a large long to int keeps only the low 32 bits.
- **Rounding rule used in this program:** the only fractional amount is the 5% tax. It is rounded **half-up** to a whole paisa with integer arithmetic `(x * 5 + 50) / 100`, and from then on everything is exact integer paise.

Places in the program where different types are combined

Where	Expression	Conversion
input.c	`*category != 'D'`	char promoted to int (integer promotion)
tariff.c	`category == 'C'`	char promoted to int
tariff.c	`subtotal * TAX_PERCENT + 50`	int constant 50 converted to long (all other operands are already long)
tariff.c	`(…) / 100`	int constant 100 converted to long; integer division (fraction discarded)
report.c	`paise / 100, paise % 100`	int constants converted to long
report.c	`printf("%c", category)`	char promoted to int (default argument promotion)
report.c	`energy + fixed + tax`	long + long + long, no conversion
bill.h	`RATE1_PAISE 150L, SLAB1_LIMIT 100L …`	constants written with the L suffix so the arithmetic is long from the start (no int overflow)

Procedure / Algorithm

1. Read the program and list every expression that mixes types (table above).
2. Write conv.c containing ten expressions where conversion changes the result; run it.
3. Explain each unexpected result using the conversion rules.
4. Compare tax rounding by truncation, half-up and round() in rnd.c.
5. Fix the rule: tax is rounded half-up once, in integer paise.
6. Measure float-versus-integer error for n bills in err.c.

Program / Source Code

conv.c

```
#include <stdio.h> int main(void)
{
    long  units = 250;
```

```
    int   a = -1;
unsigned int b = 1;     long
big = 5000000000L;     float
f = 16777217.0f;

    printf("1) units / 100            = %ld        (expected 2.5)\n", units / 100);     printf("2)
units / 100.0          = %.2f     (one operand double)\n", units / 100.0);     printf("3) (units /
100) * 150    = %ld      (expected 375)\n", (units / 100) * 150);     printf("4) -1 < 1u
= %d          (expected 1)\n", a < b);     printf("5) (int)5000000000L       = %d\n", (int)big);
printf("6) (long)(0.29 * 100)     = %ld          (expected 29)\n", (long)(0.29 * 100));
printf("7) (long)(0.29*100 + 0.5) = %ld          (rounded)\n", (long)(0.29 * 100 + 0.5));
printf("8) 0.1 + 0.2 == 0.3       = %d           (expected 1)\n", 0.1 + 0.2 == 0.3);     printf("9)
float 16777217         = %.0f\n", f);     printf("10) 7 / 2 = %d,  7 / 2.0 = %.1f,  (double)7 / 2
= %.1f,  (double)(7 / 2) = %.1f\n",
           7 / 2, 7 / 2.0, (double)7 / 2, (double)(7 / 2));
return 0;
}
```

rnd.c

```
#include <stdio.h>
#include <math.h> int main(void){     long sub[] = {1010L, 1030L, 1050L, 62500L, 4500L};
printf("subtotal   5%% exact   truncate   half-up(int)   round()\n");    for(int i=0;i<5;i++){
double ex = sub[i]*5/100.0;      printf("%7ld    %8.2f   %8ld   %12ld   %8.0f\n", sub[i], ex,
(long)ex, (sub[i]*5+50)/100, round(ex)); }   return 0; }
```

Output Screenshot

(a) Conversion surprises

```
$ gcc -Wall -Wextra -o conv conv.c
13:72: warning: comparison of integer expressions of different signedness: 'int' and 'unsigned int' [-Wsign
compare]
$ ./conv
1)  units / 100            = 2         (expected 2.5)
2)  units / 100.0          = 2.50      (one operand double)
3)  (units / 100) * 150    = 300       (expected 375)
4)  -1 < 1u                = 0           (expected 1)
5)  (int)5000000000L       = 705032704
6)  (long)(0.29 * 100)     = 28          (expected 29)
7)  (long)(0.29*100 + 0.5) = 29          (rounded)
8)  0.1 + 0.2 == 0.3       = 0           (expected 1)
9)  float 16777217         = 16777216
10) 7 / 2 = 3,  7 / 2.0 = 3.5,  (double)7 / 2 = 3.5,  (double)(7 / 2) = 3.0
```

(b) Rounding rules for the 5% tax

```
$ ./rnd   (tax in paise)
subtotal  5% exact  truncate  half-up(int)  round()    1010      50.50        50           51        51
   1030      51.50        51            52        52
   1050      52.50        52            53        53
  62500    3125.00      3125          3125      3125
   4500     225.00       225           225       225
```

Analysis / Discussion

Expression	Output	Explanation
`units / 100 (units = 250)`	2	Both operands are integers, so integer division discards the .5. Using 100.0 gives 2.50.
`(units / 100) * 150`	300 (not 375)	The fraction is lost *before* the multiplication. This is where integer division gives a wrong result. Multiply first or use paise: units * 150 / 100.
`-1 < 1u`	0 (false)	int and unsigned int meet: -1 is converted to unsigned = 4294967295, which is larger than 1. gcc -Wextra warns (-Wsign-compare).
`(int)5000000000L`	705032704	The cast keeps only the low 32 bits: 5000000000 mod 2^{32}. The value is silently corrupted.
`(long)(0.29 * 100)`	28 (not 29)	0.29 cannot be stored exactly; 0.29 * 100 is 28.999999999999996 and the cast truncates it. Adding 0.5 before the cast gives 29.
Expression	**Output**	**Explanation**

`0.1 + 0.2 == 0.3`	0 (false)	None of the three values is exact in binary; the sum differs from 0.3 in the last bit.
`float 16777217`	16777216	A float has a 24-bit mantissa; integers above 2^{24} are no longer all representable.

Rounding rule. In table (b) truncation loses up to one paisa on every bill (50.50 becomes 50), while half-up gives 51. The program therefore always uses `(x * 5 + 50) / 100`. The rule is applied in exactly one function, tax_paise(), so every bill is rounded in the same way.

Alternative Concept and Complexity Analysis — all calculations in float, rounded once at the end

Solution. Keep every amount in float (rupees: 1.5, 2.5, 4.0, 0.05), add everything up and call `lround(total * 100)` only once at the end. err.c compares this with the integer-paise totals for n bills; units of the i-th bill are (i mod 300) + 1.

```
$ ./err
   n bills    exact (paise)    float total   double total    float err   double err         10
34915          34913          34913           -2           -2
     1000         33471550       33471375       33471375         -175         -175
   100000       3598973050     3598830000     3598956375      -143050       -16675
  1000000      36012623050    36009884800    36012456375     -2738250      -166675   10000000
360149123050   357036851200   360147456375  -3112271850     -1666675

time for 1e8 bill calculations:  integer paise = 0.22 s,  float rupees = 0.30 s space per value: long =
8 bytes, float = 4 bytes, double = 8 bytes
```

Error growth. There are two separate effects:

- **Rounding policy.** A real bill is printed and paid in whole paise, so each bill is rounded individually. Rounding only once at the end cannot reproduce that. The double column shows this: error −175, −16675, −166675, −1666675 for n = 10^3–10^7, i.e. a steady ≈0.17 paisa per bill, **growing linearly, O(n)**.
- **Binary representation (float only).** A float has about 7 significant digits. Once the running total is large, a small bill is rounded off when it is added. The float error is 143050 paise at n = 10^5 and over 3×10^9 paise (about Rs 31 million, 0.9% of the total) at n = 10^7 — worse than linear.
- **Integer paise** has zero error by construction; it only needs a long wide enough for the total (a 32-bit long overflows at about Rs 21 million).

	Integer paise	**float rupees**
Time per value	O(1); 0.22 s for 10^8 values	O(1); 0.30 s for 10^8 values
Space per value	O(1); long = 8 bytes	O(1); float = 4 bytes (double = 8)
Error in total of n bills	0	grows with n (linear for double, faster for float)

Result

All places where types are mixed were identified, seven surprising expressions were shown with explanations, and one consistent rounding rule (half-up, in integer paise) was adopted. I learned that integer paise is as fast and as small as float but exact, and that float errors grow with the number of bills.

TASK 4 — Trace the Compilation Process

4 Marks | Bloom's Level: K3 – Apply

Task / Problem Statement

Follow the bill program through all the stages of compilation, show the output of each stage, explain what the preprocessor removed or replaced, classify compile-time, link-time and run-time errors with an example of each, and explain the warnings given by `gcc -Wall -Wextra`.

Aim

To understand what happens between source code and executable, and to read and classify the messages the tools produce.

Concept / Theory

Stage	Command	Output	What happens
1 Preprocessing	`gcc -E`	bill.i	Headers are pasted in, macros replaced, comments removed, #ifdef/#ifndef decided
2 Compilation	`gcc -S`	bill.s	C is checked (syntax, types) and translated to assembly language
3 Assembly	`gcc -c`	bill.o	Assembly becomes machine code in a relocatable object file; calls to other files are left as undefined symbols
4 Linking	`gcc bill.o`	executable	The linker joins object files and libraries and fixes every symbol address

- **Compile-time error:** the compiler front-end finds a language mistake; no object file is produced.
- **Link-time error:** every file compiles, but the linker cannot find the definition of a function or variable (*undefined reference*).
- **Run-time error:** the program builds without any message but fails while running (division by zero, bad pointer).
- **Warnings** (-Wall -Wextra) are not errors: the program builds, but the code is probably wrong.

Procedure / Algorithm

1. Run `gcc -E bill.c -o bill.i` and compare bill.c with bill.i.
2. Run `gcc -S bill.c -o bill.s` and read the assembly of one function.
3. Run `gcc -c bill.c -o bill.o` and inspect it with `file` and `nm`.
4. Link with `gcc bill.o -o bill_exe` and inspect the executable.
5. Compile a draft with `-Wall -Wextra`, read the warnings and fix them.
6. Produce one compile-time, one link-time and one run-time error on purpose.
7. Build the program from separate files with a Makefile and measure the build.

Program / Source Code

***draft.c** — first draft, compiled with -Wall -Wextra*

```
#include <stdio.h>
#include "bill.h"

int main(int argc, char *argv[])
{
    long  units  =  250;
int  count = 3;     int
extra;     unsigned int
n = 5;

    printf("Units = %d\n", units);
if (count < n)         printf("count
is smaller\n");     return 0;
}
```

***draft_fixed.c** — after fixing the warnings*

```
#include <stdio.h>
#include "bill.h"

int main(void)
{
    long units = 250;
int  count = 3;
unsigned int n = 5;

    printf("Units = %ld\n", units);
if ((unsigned int)count < n)
printf("count is smaller\n");
return 0;
}
```

***Makefile** — separate compilation*

```
CC = gcc
```

```
CFLAGS = -Wall -Wextra
OBJS = main.o input.o tariff.o report.o bill: $(OBJS)
    $(CC) $(OBJS) -o bill
%.o: %.c bill.h
    $(CC) $(CFLAGS) -c $< -o $@
```

Output Screenshot

(a) Stage 1: preprocessor output

```
$ gcc -E bill.c -o bill.i
bill.c : 118 lines   ->   bill.i : 967 lines

$ grep -n "energy_charge_paise" bill.i | head
821:long energy_charge_paise(long units);
894:long energy_charge_paise(long units)
962:    energy = energy_charge_paise(units);

$ sed -n "/^long energy_charge_paise(long units)$/,/^}/p" bill.i   (macros replaced) long
energy_charge_paise(long units)
{
    long charge = 0, s;

    s = (units < 100L) ? units : 100L;     charge +=
s * 150L;     if (units > 100L) {         s =
((units < 200L) ? units : 200L) - 100L;
charge += s * 250L;
    }
    if (units > 200L)
        charge += (units - 200L) * 400L;

    ((void)0);
return charge;
}
```

(b) Stage 2: assembly output

```
$ gcc -S bill.c -o bill.s
bill.s : 597 lines of assembly

$ sed -n "/^energy_charge_paise:/,/ret/p" bill.s | head -24 energy_charge_paise:
.LFB2:
    .cfi_startproc
endbr64     pushq
%rbp
    .cfi_def_cfa_offset 16
    .cfi_offset 6, -16
movq    %rsp, %rbp
    .cfi_def_cfa_register 6
movq    %rdi, -24(%rbp)
movq    $0, -16(%rbp)
movq    -24(%rbp), %rax
movl    $100, %edx     cmpq
%rdx, %rax     cmovg    %rdx,
%rax     movq    %rax, -
8(%rbp)     movq    -8(%rbp),
%rax     imulq    $150, %rax,
%rax     addq    %rax, -
16(%rbp)     cmpq    $100, -
24(%rbp)
```

```
    jle    .L18     movq
-24(%rbp), %rax     movl
$200, %edx     cmpq
%rdx, %rax
```

(c) Stages 3 and 4: object file and executable

```
$ gcc -c bill.c -o bill.o
$ file bill.o
bill.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped
$ nm bill.o
                 U __isoc99_scanf
                 U __stack_chk_fail
0000000000000220 T energy_charge_paise
                 U feof
00000000000002bd T fixed_charge_paise
0000000000000000 t flush_line
                 U fprintf
                 U fwrite
                 U getchar
```

```
000000000000054d T main
00000000000003c6 T print_bill
0000000000000328 T print_paise
                 U printf
                 U putchar
                 U puts
0000000000000024 T read_record
                 U stderr
                 U stdin
00000000000002de T tax_paise

$ gcc bill.o -o bill_exe
$ file bill_exe
bill_exe: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter
    /lib64/ld-l
$ ls -l bill.c bill.i bill.s bill.o bill_exe
    3530  bill.c
   25147  bill.i
    6224  bill.o
   10257  bill.s
   16648  bill_exe
```

(d) Warnings from -Wall -Wextra, and the fixed version

```
$ gcc -Wall -Wextra -c draft.c
11:22: warning: format '%d' expects argument of type 'int', but argument 2 has type 'long int' [-Wformat=]
12:15: warning: comparison of integer expressions of different signedness: 'int' and 'unsigned int' [-Wsign
compare]
8:10: warning: unused variable 'extra' [-Wunused-variable]
4:14: warning: unused parameter 'argc' [-Wunused-parameter]
4:26: warning: unused parameter 'argv' [-Wunused-parameter]

$ gcc -Wall -Wextra -c draft_fixed.c
(no warnings)
```

(e) One error of each kind

```
--- COMPILE-TIME error (missing semicolon) ---
$ gcc -c e_compile.c e_compile.c:
In function 'main':
e_compile.c:5:5: error: expected ',' or ';' before 'printf'
    5 |     printf("%ld\n", units);
      |     ^~~~~~

--- LINK-TIME error (tariff.c / report.c left out of the link) ---
$ gcc main.c input.c -o app
main.c:(.text+0x69): undefined reference to `energy_charge_paise'
/usr/bin/ld: main.c:(.text+0x7b): undefined reference to `fixed_charge_paise'
/usr/bin/ld: main.c:(.text+0x92): undefined reference to `tax_paise'
/usr/bin/ld: main.c:(.text+0xc4): undefined reference to `print_bill' collect2:
error: ld returned 1 exit status

--- RUN-TIME error (division by zero) --$
gcc -o e_run e_run.c   (compiles cleanly)
$ ./e_run
Floating point exception
exit status: 136
```

Analysis / Discussion

What the preprocessor removed or replaced

- The 118-line bill.c became a 967-line bill.i: about 850 lines came from **#include <stdio.h>** being pasted in; the `#include "bill.h"` line was replaced by the contents of bill.h.
- All **comments** were removed and the guard lines (#ifndef / #define / #endif) were consumed.
- **Macros were replaced:** `SLAB1_LIMIT` became `100L`, `RATE1_PAISE` became `150L`, and `DBG("...")` became `((void)0)` because DEBUG was not defined.

The later stages

- **Assembly:** the constants are already numbers in the code of energy_charge_paise (`imulq $150`, `cmpq $100`); the ternary operator became `cmovg`.
- **Object file:** `nm` shows `T` for functions defined here (energy_charge_paise, main) and `U` for undefined ones (printf, scanf, fprintf), which the linker takes from the C library. bill.o is 6224 bytes but not yet runnable.
- **Executable:** a 16648-byte dynamically linked ELF file; the startup code and the dynamic-library information are added at link time.

Errors and warnings

Kind	Example from this work	Why it appears at that stage
Compile-time	Missing ; → *expected ',' or ';' before 'printf'*	Syntax is checked by the compiler; no .o file is produced.
Link-time	main.c and input.c linked without tariff.c and report.c → *undefined reference to energy_charge_paise*	Each file compiles because the prototype in bill.h is enough; only the linker looks for the real definition.
Run-time	units / days with days = 0 → *Floating point exception*, exit status 136 (128 + signal 8)	Compiler and linker cannot know the value of days; the CPU traps while running.

Warning from -Wall -Wextra	What it told me	How I resolved it
-Wformat: %d expects int, argument is long	Wrong specifier; prints a wrong value for large units	Changed to %ld
-Wsign-compare: int and unsigned int	-1 would be converted to a huge unsigned value	Cast count to unsigned int after knowing it is non-negative
-Wunused-variable: extra	Declared but never used; probably dead or forgotten code	Removed the variable
-Wunused-parameter: argc, argv	main does not use its arguments	Changed to int main(void)

Alternative Concept and Complexity Analysis — separate compilation

Build process. The program is split into input.c, tariff.c, report.c and main.c, all including bill.h. Each .c file is compiled on its own to a .o file `(gcc -c)` and then all .o files are linked. The Makefile compares time-stamps: after a change to tariff.c only tariff.o is rebuilt and the program relinked; if bill.h changes, all four are rebuilt.

Build time (average of 50 runs)

```
full build  : single file  (gcc bill.c)                          53771 us full
build  : split (4 x gcc -c + link)                              123373 us after
editing tariff.c : single file (recompile everything)          59354 us after
editing tariff.c : split (tariff.o only + link)                53095 us
```

Space used by the object files

```
$ size x.o m.o i.o t.o r.o
   text       data        bss        dec        hex    filename
   2440          0          0       2440        988    x.o
    355          0          0        355        163    m.o
    887          0          0        887        377    i.o
    416          0          0        416        1a0    t.o     948          0          0        948        3b4

r.o sum of split .o text = 1588 bytes;  single .o text = 1588 bytes

$ ls -l *.o
  3160  i.o
  2160  m.o
  3216  r.o
  1680  t.o
  6224  x.o
```

	Single file	Separate files
Compile time after changing one file	O(N): the whole program of N lines is compiled again	O(n + link): only the changed file of n lines, then relink
Measured (118 lines)	≈ 54–59 ms	≈ 53 ms after the change, but 123 ms for the first full build (4 compiler runs)
Object-file space	6224 bytes	3160 + 2160 + 3216 + 1680 = 10216 bytes
Machine code (.text)	1588 bytes	1588 bytes — identical

Conclusion. For a program this small, starting the compiler (about 25 ms) dominates, so the two methods take about the same time, and the first full build is even slower when split. The advantage grows with the program size: in a project of 100,000 lines, changing a 500-line file recompiles about 0.5% of the code instead

of all of it. The price is space: the machine code is identical, but each object file has its own headers and symbol tables, so the object files total about 4 KB (64%) more here.

Result

The program was traced through preprocessing, compilation, assembly and linking, the effect of the preprocessor was shown, and one example of each error type and four warnings were explained and fixed. I learned to tell at which stage a message comes from and how separate compilation speeds up rebuilding large programs.

Test Cases / Validation Table

Each test was run on the compiled program (`./bill`) and the result compared with the value calculated by hand from the tariff.

No.	Input / Scenario	Expected Output	Actual Output	Status
1	D, 0 units	TOTAL Rs 26.25	TOTAL Rs 26.25	PASS
2	D, 100 units (end of slab 1)	TOTAL Rs 183.75	TOTAL Rs 183.75	PASS
3	D, 101 units (first unit of slab 2)	TOTAL Rs 186.38	TOTAL Rs 186.38	PASS
4	D, 250 units (all three slabs)	TOTAL Rs 656.25	TOTAL Rs 656.25	PASS
5	C, 120 units (commercial)	TOTAL Rs 315.00	TOTAL Rs 315.00	PASS
6	D, 100000 units (maximum)	TOTAL Rs 419606.25	TOTAL Rs 419606.25	PASS
7	Units "abc", then 120	Error message, then TOTAL Rs 236.25	Error message, then TOTAL Rs 236.25	PASS
8	Units "abc", "xyz", "pqr"	Error x3, program ends, exit 1	Error x3, program ends, exit 1	PASS
9	Units -5 three times	Out-of-range error x3, exit 1	Out-of-range error x3, exit 1	PASS
10	Category X	Category error, exit 1	Category error, exit 1	PASS
11	Consumer number "abc"	Error, exit 1	Error, exit 1	PASS

Final Submission Checklist

- ☐ Student and course details completed
- ☑ Task statement, aim, theory, and algorithm included
- ☑ Program code is readable and runs successfully
- ☑ Output screenshot included
- ☑ Analysis and result written in own words
- ☑ Test cases completed where applicable
- ☐ GitHub repository link added if required